

IRI - Integrated Research Infrastructure

John MacAuley & Justas Balcas



Genesis Mission

AMERICAN SCIENCE CLOUD



U.S. DEPARTMENT of ENERGY

IRI Interfaces Charter

- Gather requirements, define use cases, and specify unified interfaces for the DOE ASCR Facilities.
- Tasked with enhancing the "user experience" for interaction and management of complex workflows across ASCR facilities
 - Goal is to facilitate a more cohesive and efficient research infrastructure.
- Consider applicable standards and build upon the work of other organizations where available to deliver a reference IRI interface implementation.

Launched May 2024

Interfaces Technical Subcommittee

- Design a **minimal functional API** and deploy it at multiple sites
 - Review existing API schemas
 - Propose IRI schema: endpoints, architecture, infrastructure,...
 - Implement MVP
- Explore how to align **Jupyter** across sites

Co-Chairs: John MacAuley, Paul Rich

IRI Development - How and What

- The How
 - The Interfaces Subcommittee co-designs the IRI specification
 - Ideas are discussed, prototyped, and battle-tested before they land in the specification
 - Decisions are not sacred - we revisit and refine as we learn
- The What
 - HTTP/JSON APIs shaped by pragmatic REST principles
 - An OpenAPI specification serves as the formal contract and documentation
 - Reference implementations jump-start adopters and boost development velocity
 - Multiple independent implementations validate interoperability and the specification itself

IRI Facility Specification (version 1)

Facility

- Facility
- Site
- Location
- Resource



Allocation

- Project
- UserAllocaton
- ProjectAllocation
- AllocationEntry
- Capability
- Resource



Compute

- Job
- Resource



Status

- Incident
- Event
- Resource



Filesystem

- File
- Path
- Resource



V1 - Job Submission

- Used PSI/J as initial concept
- Supported operations
 - Submit
 - Live status
 - Delete/Cancel
- Workload Managers
 - SLURM REST API
 - PBS GraphQL API
 - Kube Kueue

compute ^

POST	/api/v1/compute/job/{resource_id}	Submit Job	🔒 ▼
PUT	/api/v1/compute/job/{resource_id}/{job_id}	Update Job	🔒 ▼
GET	/api/v1/compute/status/{resource_id}/{job_id}	Get Job Status	🔒 ▼
POST	/api/v1/compute/status/{resource_id}	Get Job Statuses	🔒 ▼
DELETE	/api/v1/compute/cancel/{resource_id}/{job_id}	Cancel Job	🔒 ▼

V1 - Local FileSystem Operations

- Used FirecREST as initial concept
- Local Operation Only
 - Complimentary to transfer frameworks
- Asynchronous only
- Similar to operations through SSH

filesystem			⤴
PUT	/api/v1/filesystem/chmod/{resource_id}	Put Chmod	🔒 ⌵
PUT	/api/v1/filesystem/chown/{resource_id}	Put Chown	🔒 ⌵
GET	/api/v1/filesystem/file/{resource_id}	Get File	🔒 ⌵
GET	/api/v1/filesystem/stat/{resource_id}	Get Stat	🔒 ⌵
POST	/api/v1/filesystem/mkdir/{resource_id}	Post Mkdir	🔒 ⌵
POST	/api/v1/filesystem/symlink/{resource_id}	Post Symlink	🔒 ⌵
GET	/api/v1/filesystem/ls/{resource_id}	Get Ls Async	🔒 ⌵
GET	/api/v1/filesystem/head/{resource_id}	Get Head	🔒 ⌵
GET	/api/v1/filesystem/view/{resource_id}	Get View	🔒 ⌵
GET	/api/v1/filesystem/tail/{resource_id}	Get Tail	🔒 ⌵
GET	/api/v1/filesystem/checksum/{resource_id}	Get Checksum	🔒 ⌵
DELETE	/api/v1/filesystem/rm/{resource_id}	Delete Rm	🔒 ⌵
POST	/api/v1/filesystem/compress/{resource_id}	Post Compress	🔒 ⌵
POST	/api/v1/filesystem/extract/{resource_id}	Post Extract	🔒 ⌵
POST	/api/v1/filesystem/mv/{resource_id}	Move Mv	🔒 ⌵
POST	/api/v1/filesystem/cp/{resource_id}	Post Cp	🔒 ⌵
GET	/api/v1/filesystem/download/{resource_id}	Get Download	🔒 ⌵
POST	/api/v1/filesystem/upload/{resource_id}	Post Upload	🔒 ⌵

Full Site Validation

- Facilities tested periodically:
 - API readiness
 - Full workflow lifecycle
 - Includes real filesystem and compute operations

Operational sites:

- NERSC, ALCF, ESnet (AWS)

Work in progress:

- ORNL, PNNL, SLAC, BNL

operationId	DemoAdapter	NERSC	ALCF	ESnet_West	ESnet_East	ORNL	PNNL	SLAC	BNL
GET /api/v1/facility	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/facility/sites	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/facility/sites/{site_id}	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/status/events	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/status/events/{event_id}	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/status/incidents	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/status/incidents/{incident_id}	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/status/resources	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/status/resources/{resource_id}	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/account/capabilities	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/account/capabilities/{capability_id}	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/account/projects	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/account/projects/{project_id}	PASS	PASS	PASS	PASS	PASS				
GET /api/v1/account/projects/{project_id}/project_allocations	PASS	PASS	PASS	PASS	PASS				

Work in Progress

Roadmap to V2: Static resource specification

Enhance IRI resource descriptions and align with AmSC resource cards to allow for a consistent discovery strategy. This includes (mostly) static information such as:

- More detailed definition of a resource:
 - Compute (Node, CPU, GPU, container) + capacity, limits, and queuing policies.
 - Storage (direct connect, NFS) + capacity, mounts, limits, retention policies.
 - DTN (Globus, XRootD) endpoints.
 - Networking connectivity, and other service capabilities (inference).
 - Flexible resource definitions with resource specific extensions.
- CUI capabilities for resources.
- Facility topology with connectivity modelling (DTN <-> storage <-> compute)

This would allow workflows to determine job capabilities and associated policies at each facility, storage and transfer capabilities, as we as determine which DTN endpoints are used to transfer data to/from storage associated with a specific compute resource.

Roadmap to V2: Dynamic resource specification

In support of workflow/resource orchestrations will add situational awareness of resources at a Facility. This includes real-time visibility into:

- Current compute resource capacities (Total/InUse/Idle/Min/Max CPU, GPU), job queue access, and wait times.
- Storage resource capacities, limits for project, home, and scratch directories.

This would allow us to estimate the best location for job runtime, forward jobs to correct facilities, and optimize delivery time.

* https://docs.google.com/document/d/1ou1-lhKXxysDZJYoT1XCsuBj1D5bhTezCRll_IMclk/edit?tab=t.0

* <https://github.com/doe-iri/iri-facility-api-docs/issues/new?template=newapi.yml>

Roadmap to V2: Additional capabilities

- **Compute:**

- Container capabilities, container upload
- Normalized job submission
- Custom Job Scripts
- Inbound/outbound communications

- **Filesystem:**

- Logical file name/directory normalization

- **Other:**

- Ping and auth modes supported
- AmSC project to local allocation
- Service/Robot accounts
- Persistent services
- Integrated file transfer

Pre-Recorded Demos

https://drive.google.com/drive/folders/1SWtfeB_-fjSQJNwdQNnwvVVsOEaRah9b?usp=sharing



Chat with IRI Endpoints (Beta)

Submit an IRI compute job with the following configuration:
Resource: esnet-east (fb0aafe1-c780-55c0-b635-a7121f1b0ce5)
Run a bash login shell in /data/home/jbalcas that:

- Prints system info (time, hostname, user)
- Dumps environment variables (sorted)
- Writes a log file to /data/home/jbalcas/esnet_iri_test.log

Capture stdout and stderr into:

- /data/home/jbalcas/esnet_iri_test_stdout.log
- /data/home/jbalcas/esnet_iri_test_stderr.log

Resources:

- 1 node, 1 process, 1 CPU core
- 256MB memory
- Exclusive node usage

Scheduler settings:

- Queue: debug
- Account: interactive
- Max duration: 600 seconds

I'll submit that job to the ESnet East resource for you: Perfect! Your job has been successfully submitted to ESnet East. Here are the details:

Job Submission Success

Job ID: 8a935d40-17cf-4d47-a070-24920da56fb1

Resource ID: fb0aafe1-c780-55c0-b635-a7121f1b0ce5

Status: New

Job Name: iri-test-job-20260326-150708

The job is a smoke test that will:

Display system information (date, hostname, user)

Output environment variables

Create a log file at /data/home/jbalcas/

Here's the facility information for NERSC:

National Energy Research Scientific Computing Center (NERSC)

Description: NERSC is the mission science computing facility for the U.S. Department of Energy Office of Science, the nation's largest supporter of basic research in the physical sciences.

Short Name: NERSC

NERSC is a major computing center that provides high-performance computing resources for scientific research funded by the Department of Energy. Would you like more information about NERSC's resources, capabilities, or other details?

Get job status on ESnet East for Job ID: 8a935d40-17cf-4d47-a070-24920da56fb1
Resource ID: fb0aafe1-c780-55c0-b635-a7121f1b0ce5
Job Name: iri-test-job-20260326-150708

I'll get the job status for you on ESnet East. Perfect! Here's the job status for **iri-test-job-20260326-150708**:

Job Status Summary:

Job ID: 8a935d40-17cf-4d47-a070-24920da56fb1

Resource ID: fb0aafe1-c780-55c0-b635-a7121f1b0ce5

Site: ESnet East

Current State: ● ACTIVE/RUNNING

Exit Code: -1 (still running)

Start Time: 2026-04-06T14:30:42Z

Completion Time: Not yet completed

Metadata:

Active: 1

Succeeded: 0

Failed: 0

The job is currently running on ESnet East and has been active since April 6, 2026 at 14:30:42 UTC.

Summary

DOE-IRI: <https://github.com/orgs/doe-iri/repositories> (api-docs, api-ref, api-examples, api-toolkit)

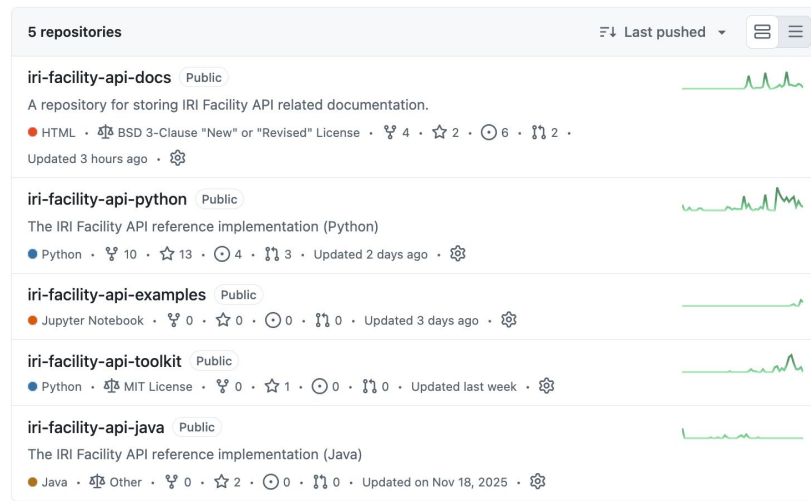
New API Requests: <https://github.com/doe-iri/iri-facility-api-docs/issues/new?template=newapi.yml>

V2 Requests in Progress

3 Facilities fully compliant with V1 Specification

4 Facilities finalizing V1 Implementation

Link to slides: <https://shorturl.at/h6e47>



A screenshot of a GitHub page showing a list of 5 repositories under the DOE-IRI organization. The repositories are: 1. 'iri-facility-api-docs' (Public), described as 'A repository for storing IRI Facility API related documentation.', with 4 forks, 2 stars, 6 issues, and 2 pull requests, updated 3 hours ago. 2. 'iri-facility-api-python' (Public), described as 'The IRI Facility API reference implementation (Python)', with 10 forks, 13 stars, 4 issues, and 3 pull requests, updated 2 days ago. 3. 'iri-facility-api-examples' (Public), described as 'Jupyter Notebook', with 0 forks, 0 stars, 0 issues, and 0 pull requests, updated 3 days ago. 4. 'iri-facility-api-toolkit' (Public), described as 'The IRI Facility API reference implementation (Python)', with 0 forks, 1 star, 0 issues, and 0 pull requests, updated last week. 5. 'iri-facility-api-java' (Public), described as 'The IRI Facility API reference implementation (Java)', with 0 forks, 2 stars, 0 issues, and 0 pull requests, updated on Nov 18, 2025. Each repository entry includes a green line graph showing activity over time.

 Genesis Mission | AMERICAN SCIENCE CLOUD

